SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Real-Time Chat Application

Team 5 | Version 1.1 | 09 October 2026

Field	Details
Project	Real-Time Chat Application
Document	Software Architecture and Design (SAD)
Version	1.1 — revised draft
Team	Team 5
Authors	Amoghaa R, Vaibhav R, P S Harshavardhan, Pramath Ganesh Hegde
Status	Draft for team review
Implementation note	This document describes the current Flask-based prototype separately from features planned for later sprints.

Revision History

Version	Date	Author	Change summary
1.0	09-10-2026	Team 5	Initial architecture draft based on the supplied SAD template and project documents.
1.1	09-10-2026	Team 5	Reworded for clarity, aligned with the working Flask prototype, and separated current functionality from planned features.

Approvals

Role	Name	Approval / signature	Date
Project Guide / Course Coordinator	To be completed	Pending	—
Team Representative	To be confirmed	Pending	—

Contents

- 1. Introduction
- 2. Document Overview
- 3. Architecture
- 4. Design
- 5. Appendices

1. Introduction

1.1 Purpose

This document explains how Team 5 has organised the Real-Time Chat Application and how its main parts work together. It is intended to help the team implement the project consistently, explain the design during reviews, and check later changes against the requirements.

1.2 Scope

The current prototype lets a user enter a username, send a message, and see messages from another browser tab. The browser communicates with a Python Flask server using HTTP requests. The page periodically asks the server for new messages, so updates appear without manually refreshing the page.

Features such as user accounts, private conversations, group chats, online presence, typing indicators, file sharing, persistent chat history, moderation, and WebSocket-based messaging are planned possibilities from the wider requirements. They should not be treated as completed unless the team implements and tests them.

1.3 Audience

The main readers are the project team, course coordinator or faculty, and anyone who needs to review, test, or maintain the application.

1.4 Definitions

Term	Meaning
Client	The chat page opened in a browser.
Server	The Python application that receives requests and returns message data.
HTTP	The request-and-response protocol used by the current prototype.
Polling	The browser requests the latest messages at regular intervals.
API endpoint	A URL on the server that performs a defined operation.
SAD / SRS / STP	Software Architecture and Design / Software Requirements Specification / Software Test Plan.

2. Document Overview

2.1 How to use this document

Sections 3 and 4 describe the current architecture, its responsibilities, request flows, interfaces, and known limitations. The final section lists references and items that the team should confirm before calling this document final.

2.2 Related documents

- Software Requirements Specification (SRS) for the Real-Time Chat Application.
- Software Test Plan (STP) for the Real-Time Chat Application.
- GitHub repository, issue backlog, and sprint plan.

2.3 Design approach

- Keep the browser interface separate from the server-side message handling.
- Use simple HTTP endpoints for the current prototype so the team can test the basic flow easily.
- Validate incoming requests and handle errors without exposing internal server details.

- Describe planned functionality as planned; update this document when the implementation changes.

3. Architecture

3.1 Goals and constraints

Goal / constraint	Current design response
Simple real-time-style chat	The browser requests the current message list repeatedly; this is polling, not a WebSocket connection.
Easy to run locally	The application runs using Python and Flask on a local development address.
Readable code	The browser code handles the form and rendering, while Flask handles HTTP routes and message storage.
Prototype limitation	Messages are held in server memory and may be lost when the server restarts.
Project timeline	The current version demonstrates the basic message exchange; larger features need separate implementation and testing.

3.2 Stakeholders and concerns

Stakeholder	Main concern
Chat user	Can enter a username, send a message, and see messages from other open clients.
Development team	Keep the code understandable and make it possible to add features incrementally.
Tester / reviewer	Verify that messages can be sent and received in two browser tabs and record any failures.
Course coordinator / faculty	Check that the submitted design and demo match the actual state of the project.

3.3 High-level component architecture

Component	Role in the current prototype
Browser interface (index.html)	Displays the username field, message field, send form, and message list.
Client script (client.js)	Submits messages to the server, requests the latest messages, and updates the page.
Stylesheet (style.css)	Controls the appearance and layout of the chat page.
Flask application (app.py)	Serves the page and handles the message-related HTTP endpoints.
In-memory message list	Temporarily stores messages while the Flask process is running.

Request flow: Browser page → Flask HTTP endpoint → in-memory message list → HTTP response → browser updates the displayed messages.

3.4 Component descriptions

The browser sends a message when the user submits the form. The Flask server receives it through the send endpoint and adds it to the in-memory list. The browser periodically requests the message list and redraws the messages it receives. Because the current storage is in memory, this version does not provide durable chat history.

3.5 Architecture pattern and rationale

The prototype uses a small client-server design. Flask handles server-side requests, while HTML, CSS, and JavaScript handle the user interface. This approach was chosen because it is straightforward to run and demonstrate for the initial version. Polling is easier to set up than a persistent socket connection, although it can generate repeated requests and is not as immediate or efficient as a properly implemented WebSocket solution.

3.6 Technology stack and data storage

Area	Technology / approach	Status
Server	Python with Flask	Used in the current prototype.
Client	HTML, CSS, JavaScript	Used in the current prototype.
Communication	HTTP requests using fetch	Used for sending and retrieving messages.
Message updates	Periodic polling	Used; no Socket.IO/WebSocket implementation in the current prototype.
Storage	In-memory Python list	Temporary; messages are not preserved across server restarts.
Database	Not yet connected	Choose and document one if persistent accounts/history are implemented.
Testing	Manual check in two browser tabs	Automated and load testing remain future work unless separately completed.

3.7 Risks and mitigations

Risk	Possible impact	Practical mitigation
Server stops or restarts	The page cannot fetch messages and in-memory history is cleared.	Keep the server terminal running during a demo; add persistent storage in a later iteration.
Repeated polling	More HTTP requests are made than strictly necessary.	Tune the polling interval or consider WebSockets if required by the final design.
Invalid or empty input	Unhelpful or blank messages may be sent.	Validate the username and message on the client and server.
Prototype scope is mistaken for final scope	Documentation may claim features that are not implemented.	Keep this SAD aligned with the actual code and mark future features clearly.
No authentication or persistent storage	The prototype is not suitable for private or production use.	Add authentication, authorization, and a database before treating it as a multi-user service.

3.8 Traceability to requirements

The basic message exchange is the part demonstrated by the current prototype. Other requirement groups from the SRS—such as account management, groups, presence, file sharing, persistent history, moderation, security controls, and load targets—must be mapped to implemented code and test evidence as the team completes them. A design description alone does not mean a requirement has been delivered.

Requirement area	Current status
Open chat page and enter username	Available in the prototype.
Send a message	Available through the Flask send endpoint.
Retrieve and display messages	Available through the message-list endpoint and browser polling.
Two-client message exchange	Manually demonstrated in two browser tabs.
Accounts, private/group chat, attachments, moderation	Not part of the currently demonstrated baseline; verify against the backlog.
Durable history, performance targets, production security	Not established by the current prototype; requires

	implementation and evidence.

3.9 Security considerations

The current prototype is a classroom demonstration, not a production messaging service. It does not currently establish user identity or enforce private-conversation permissions. Before expanding it, the team should validate all inputs on the server, avoid exposing internal errors, add authentication and authorization where needed, and use HTTPS for deployment. Do not describe these controls as implemented until they have been added and tested.

3.10 Availability and performance

The current version depends on the Flask process remaining available. A browser tab can display updated messages while it can reach the server, but the prototype has not been shown to meet a specific latency, concurrent-user, or availability target. Those targets should be measured using repeatable tests before being reported as achieved.

4. Design

4.1 Design overview

The main interaction has three steps: the user submits a message in the browser, Flask records it in memory, and the browser's next polling request retrieves the updated message list. The page then displays the returned messages.

4.2 Principal interaction flows

4.2.1 Sending a message

Step	Actor	Action
1	User / browser	Enters a username and message, then submits the form.
2	Client script	Sends the message data to the Flask send endpoint using fetch.
3	Flask server	Reads the submitted data and adds the message to the in-memory list.
4	Client script	Handles the server response and continues polling for the latest messages.
5	Flask server	Returns the current message list when the browser requests it.
6	Browser	Updates the message area so the new message is visible.

4.2.2 Receiving messages in another browser tab

Step	Actor	Action
1	Second browser tab	Periodically requests the current message list.
2	Flask server	Returns the messages currently held in memory.
3	Client script	Reads the response and updates the page.
4	User	Sees the message without manually refreshing the page.

4.3 HTTP endpoint design

Endpoint	Method	Purpose	Notes
/messages	GET	Return the messages currently held by the server.	Used by the browser polling loop.
/send	POST	Accept a message submitted from the browser.	Used by the form submission handler.

These endpoint names describe the current prototype. Do not add proposed Socket.IO events or authentication endpoints to the implementation record unless they actually exist in the code.

4.4 Error handling, logging and monitoring

For the current prototype, test what happens when the Flask server is stopped, a request fails, or a user submits empty input. The browser should fail gracefully where possible, and the server terminal can help identify development errors. Production-grade structured logging and monitoring have not been established by the current demonstration.

4.5 User interface

The current interface is intentionally simple: the user enters a username, types a message, submits it, and sees the conversation in the message area. The main usability check is whether a person can understand the fields and send a message without extra instructions. Responsive layouts, keyboard accessibility, and additional status indicators should be tested if they are included in the final scope.

4.6 Open issues and next steps

Item	Current position	Next step
Message storage	In-memory list	Decide whether the final scope requires a database and persistent history.
Message update mechanism	HTTP polling	Keep it if acceptable for the project or plan a deliberate WebSocket migration.
Input validation	Needs verification against current code	Test empty, long, and unusual input values.
Automated tests	Not confirmed by the demo	Add tests and retain results as the project requires.
SAD / SRS / STP consistency	Must reflect the implementation	Update documents when the team makes a confirmed technology or scope decision.
Additional features	Depends on sprint backlog	Implement and test only the agreed requirements.

5. Appendices

5.1 Glossary

Term	Definition
Flask	Python web framework used by the current server.
HTTP request	A request sent by the browser to a server endpoint.
Polling	Repeatedly asking the server for updates at a chosen interval.
Endpoint	A URL through which the server exposes an operation.
In-memory storage	Data held in the running program's memory rather than saved permanently.
Prototype	An early working version used to demonstrate core behavior.

5.2 References

- Team 5, Software Requirements Specification (SRS), Real-Time Chat Application, supplied project document.
- Team 5, Software Test Plan (STP), Real-Time Chat Application, supplied project document.
- SAD_Template(2).docx supplied for this document.
- Team 5 GitHub repository and current prototype source files.

5.3 Tools

- Python and Flask for the server.
- HTML, CSS, JavaScript, and browser developer tools for the client.
- Git and GitHub for source control and project tracking.

5.4 Final review checklist

- Confirm all team member names and the document date.
- Check that the SRS and STP use the same technology choice as the implementation.
- Confirm the endpoint names and behavior against the current source code.
- Replace any future/planned feature descriptions with verified implementation details only when evidence exists.
- Add any diagrams or test evidence specifically required by the faculty.
- Obtain guide/coordinator approval before marking the document final.